

Esame Programmazione di Sistema / API Programming (Redesign in Rust)

Data: 2021-06-19 (Mapped to Rust)

THEORETICAL AND PROGRAMMING QUESTIONS

Domanda 1

Si descriva il concetto di risultato differito, come esposto dalla classe C++11 `std::future<T>` (e dai suoi equivalenti in Rust), indicando tre modi alternativi offerti dalla libreria standard (o pattern comuni in Rust) per creare un oggetto di tale tipo e illustrando, per ciascuno di essi, un possibile caso d'uso che ne evidenzia l'utilità.

Domanda 2

In un sistema operativo conforme alle specifiche Posix, cosa succede se in un processo in cui sono in esecuzione più thread, viene invocata la chiamata di sistema `fork()`? Al suo ritorno, quanti thread saranno presenti nel processo figlio e quanti nel processo padre? Che tipo di controllo o astrazione può esercitare il programmatore per gestire questo tipo di situazione in Rust?

Domanda 3

Si spieghino le differenze tra costruzione di copia (trait `Copy`) e costruzione per movimento (Move semantics) nel linguaggio Rust in termini di comportamento e di prestazioni, e si descriva un caso d'uso nel quale il movimento risulta vantaggioso rispetto alla copia (comparato al C++).

Domanda 4

La struct `SingleThreadExecutor` implementa il concetto di `ThreadPool` basato su un singolo thread incapsulato all'interno di ciascuna sua istanza. In assenza di richieste di lavoro, tale thread resta fermo senza consumare cicli macchina.

Attraverso il metodo `submit(...)` è possibile affidare ad un'istanza di `SingleThreadExecutor` un compito da eseguire. Tale compito viene inizialmente accodato e, non appena il thread incapsulato è libero, viene eseguito. Attraverso il metodo `close()` è possibile impedire l'ulteriore accodamento di compiti. Attraverso il metodo `join()` è possibile attendere che tutti i compiti ancora da svolgere siano svolti e il thread incapsulato nell'istanza termini.

Si implementi tale struct usando le funzionalità offerte dalla libreria standard di Rust (`std::sync::Mutex`, `std::sync::Condvar`), definendo tutte le parti eventualmente

manca nella definizione. Si faccia attenzione al fatto che il codice che può essere sottomesso all'esecutore è arbitrario e può contenere richieste di sottomissione di ulteriori compiti allo stesso esecutore.

```
1 pub struct SingleThreadExecutor {  
2     // Definire i campi necessari  
3 }  
4  
5 impl SingleThreadExecutor {  
6     pub fn new() -> Self { ... }  
7     pub fn submit<F>(&self, task: F) -> Result<(), &str>  
8         where F: FnOnce() + Send + 'static { ... }  
9     pub fn close(&self) { ... }  
10    pub fn join(self) { ... }  
11 }
```

ANSWERS

Domanda 1

Concetto di Risultato Differito: Il concetto di "risultato differito" rappresenta un valore (o un'eccezione/errore) che sarà disponibile in un momento futuro, come risultato di una computazione asincrona. Un oggetto "future" funge da canale di comunicazione: chiamando un metodo di attesa (es. `join()`, `recv()` o `.await`), il thread chiamante si blocca (o cede il controllo) finché il risultato non diventa disponibile. Una volta pronto, viene restituito e il canale si consuma.

Ci sono principalmente tre modi in Rust per rappresentare e creare computazioni differite:

1. **Tramite `std::thread::spawn (JoinHandle)`:** La funzione avvia un nuovo thread del sistema operativo e restituisce un `JoinHandle<T>`. Chiamare `join()` su di esso sospende il thread corrente finché il thread figlio non termina, restituendo il risultato.

Caso d'uso: Si vuole calcolare un risultato indipendente ma computazionalmente oneroso senza bloccare il thread principale.

```
1 use std::thread;
2
3 fn compute_heavy_task(x: i32) -> i32 { x * x }
4
5 fn main() {
6     // Il thread figlio calcola il risultato asincronamente
7     let handle = thread::spawn(|| compute_heavy_task(42));
8
9     // ... il thread principale puo' fare altro qui ...
10
11    // Attende la fine del calcolo e preleva il risultato
12    let result = handle.join().unwrap();
13 }
```

2. **Tramite `std::sync::mpsc::channel`:** Un canale "one-shot" può fungere da future. Il trasmettitore invia il risultato quando ha finito, e il ricevitore attende bloccandosi su `recv()`.

Caso d'uso: Un ThreadPool dove i worker elaborano compiti, e il chiamante ha bisogno di recuperare il risultato specifico di un task.

```
1 use std::thread;
2 use std::sync::mpsc;
3
4 fn main() {
5     let (tx, rx) = mpsc::channel();
6
7     thread::spawn(move || {
8         let result = 10 + 20;
9         tx.send(result).unwrap(); // Invia il risultato
10    });
11
12    // Blocca finche' il risultato non arriva sul canale
13    let result = rx.recv().unwrap();
14 }
```

3. **Tramite il trait `std::future::Future` (`async/await`):** Un `Future` descrive una computazione differita a costo zero che può essere sospesa e ripresa da un executor asincrono senza allocare nuovi thread fisici.

Caso d'uso: Applicazioni I/O bound con migliaia di connessioni concorrenti (es. web server), dove bloccare un thread fisico per ogni attesa di I/O sarebbe insostenibile.

```
1 // Esempio usando un runtime ipotetico come tokio
2 async fn fetch_data() -> i32 {
3     // Computazione asincrona non bloccante
4     100
5 }
6
7 async fn main_async() {
8     // La keyword await sospende l'esecuzione in modo cooperativo
9     // finche' il future non e' pronto
10    let result = fetch_data().await;
11 }
```

Domanda 2

Comportamento di `fork()` in un processo multi-thread (POSIX): Se in un processo in cui sono in esecuzione più thread viene invocata la chiamata di sistema `fork()`, il sistema operativo crea un nuovo processo figlio. Al ritorno della `fork()`:

- **Nel processo figlio** sarà presente **un solo thread**, che è l'esatta replica del thread che ha invocato la `fork()`. Tutti gli altri thread del processo padre non esistono nel figlio.
- **Nel processo padre** rimangono tutti i thread originali in esecuzione.

Questo comportamento genera gravi problematiche, in particolare legati allo stato delle primitive di sincronizzazione. Se un thread nel padre, diverso da quello che ha chiamato `fork()`, possedeva un mutex nel momento in cui la clonazione è avvenuta, quel mutex risulterà permanentemente "bloccato" nel processo figlio. Essendo il thread proprietario non clonato nel figlio, il mutex non verrà mai sbloccato, causando deadlock inevitabili.

Controllo del programmatore (L'approccio di Rust): Mentre POSIX offre `pthread_atfork()` per rilasciare i mutex manualmente prima della clonazione, in Rust chiamare direttamente `libc::fork` in un contesto multi-thread è intrinsecamente insicuro (unsafe). La best-practice nei sistemi operativi concorrenti è di non effettuare logica applicativa nel processo figlio dopo la `fork()`, ma invocare immediatamente `exec()` per sostituire l'immagine della memoria. Rust nasconde ed incapsula questa problematica astrattizzando la creazione dei processi figli in modo sicuro tramite la struct `std::process::Command`. Il modulo si occupa internamente di effettuare la sequenza `fork + exec` evitando deadlock su mutex utente.

Domanda 3

Costruzione di Copia (Copy) vs. Movimento (Move Semantics):

- **Copia (Copy trait):** In Rust, alcuni tipi possono implementare il trait `Copy`. Quando un tipo è `Copy`, un assegnamento clona l'intero oggetto bit a bit (shallow copy). In C++, la copia può nascondere copie profonde (deep copy) di heap tramite copy constructor invocati implicitamente (allocazione di memoria $O(N)$). In Rust, la copia implicita esiste solo per tipi semplici stack-only (es. interi, boolean); per i tipi complessi si deve chiamare esplicitamente `.clone()`.
- **Movimento (Move Semantics):** Per default, i tipi in Rust che non sono `Copy` seguono la semantica di movimento (Move) e il trasferimento di Ownership. A differenza del C++, dove il movimento implica modificare l'oggetto sorgente per resettarne i puntatori e disabilitare il distruttore, in Rust un Move è un puro `memcpy` bit a bit sul solo record di stack, affiancato al controllo in compile-time: il Borrow Checker invalida staticamente la variabile sorgente in modo che non venga mai più usata e il suo distruttore non venga invocato doppiamente. Le prestazioni del Move in Rust sono strettamente legate al tempo costante $O(1)$.

Prestazioni e Caso d'Uso: In C++98, l'assegnazione di un `vector` causava una deep-copy dell'intero vettore. Con C++11 (o in Rust, dove il Move è il default da sempre), se una funzione restituisce un contenitore come `Vec<String>` con milioni di elementi, la semantica Move garantisce prestazioni ottimali. I dati sullo heap non vengono copiati; solo i 24 byte di controllo sullo stack cambiano proprietario. Nessuna riallocazione dinamica viene invocata.

```

1 fn get_large_data() -> Vec<String> {
2     let data = vec!["A".to_string(); 1_000_000];
3     // Ritorna il vettore passandone l'ownership (Move).
4     // Solo il puntatore stack viene copiato in O(1). Nessuna
      deep-copy.
5     data
6 }
7
8 fn main() {
9     // Ricezione dell'ownership: my_data diventa il nuovo e unico
      proprietario
10    let my_data = get_large_data();
11 }

```

Domanda 4

Implementazione di SingleThreadExecutor

Implementiamo il `SingleThreadExecutor` utilizzando manualmente una `VecDeque` protetta da un `Mutex` e svegliata da una `Condvar`.

L'aspetto critico evidenziato nel testo è l'evitare il **deadlock** quando un task sottomette a sua volta un altro task o chiama `close()`. Per evitare un auto-deadlock, l'esecutore **non deve trattenere il lock del mutex mentre esegue il task**. Per farlo, in Rust utilizziamo `drop(state)` per sbloccare la `Mutex` prima di invocare il task.

```

1 use std::collections::VecDeque;
2 use std::sync::{Arc, Condvar, Mutex};
3 use std::thread::{self, JoinHandle};
4
5 type Task = Box<dyn FnOnce() + Send + 'static>;
6

```

```

7 struct SharedState {
8     queue: VecDeque<Task>,
9     closed: bool,
10 }
11
12 pub struct SingleThreadExecutor {
13     shared: Arc<(Mutex<SharedState>, Condvar)>,
14     worker: Option<JoinHandle<()>>,
15 }
16
17 impl SingleThreadExecutor {
18     pub fn new() -> Self {
19         let shared = Arc::new((
20             Mutex::new(SharedState {
21                 queue: VecDeque::new(),
22                 closed: false,
23             }),
24             Condvar::new(),
25         ));
26
27         let shared_clone = Arc::clone(&shared);
28         let worker = thread::spawn(move || {
29             let (lock, cvar) = &*shared_clone;
30             loop {
31                 // Preleva il lock all'inizio di ogni iterazione
32                 let mut state = lock.lock().unwrap();
33
34                 // Attende se la coda e' vuota e l'executor non e'
35                 // chiuso
36                 while state.queue.is_empty() && !state.closed {
37                     state = cvar.wait(state).unwrap();
38                 }
39
40                 // Estraiamo il task dalla coda
41                 if let Some(task) = state.queue.pop_front() {
42                     // CRITICO: Rilasciare il lock PRIMA di eseguire
43                     // il task!
44                     // In questo modo, se il task invoca 'submit' o
45                     // 'close',
46                     // potra' acquisire nuovamente il lock senza
47                     // bloccarsi.
48                     drop(state);
49
50                     // L'esecuzione avviene senza possedere il lock.
51                     task();
52                 } else if state.closed {
53                     // Se la coda e' vuota e lo stato e' chiuso,
54                     // terminiamo l'iterazione e la vita del thread
55                     // worker.
56                     break;
57                 }
58             }
59         });
60
61         Self {
62             shared,
63             worker: Some(worker),
64         }
65     }
66 }

```

```

60 }
61
62 pub fn submit<F>(&self, f: F) -> Result<(), &'static str>
63 where
64     F: FnOnce() + Send + 'static,
65 {
66     let (lock, cvar) = &*self.shared;
67     let mut state = lock.lock().unwrap();
68
69     if state.closed {
70         return Err("Executor is closed");
71     }
72
73     // Accodamento e notifica del thread incapsulato se e'
74     // dormiente.
75     state.queue.push_back(Box::new(f));
76     cvar.notify_one();
77     Ok(())
78 }
79
80 pub fn close(&self) {
81     let (lock, cvar) = &*self.shared;
82     let mut state = lock.lock().unwrap();
83     state.closed = true;
84     // Notifica il worker nel caso sia bloccato sulla cvar (coda
85     // vuota)
86     cvar.notify_one();
87 }
88
89 pub fn join(mut self) {
90     // Estrae il JoinHandle dalla Option (sostituendolo con None)
91     // in modo da poterne chiamare il metodo che ne consuma
92     // l'ownership
93     if let Some(handle) = self.worker.take() {
94         handle.join().unwrap();
95     }
96 }
97
98 }
99
100 }

```